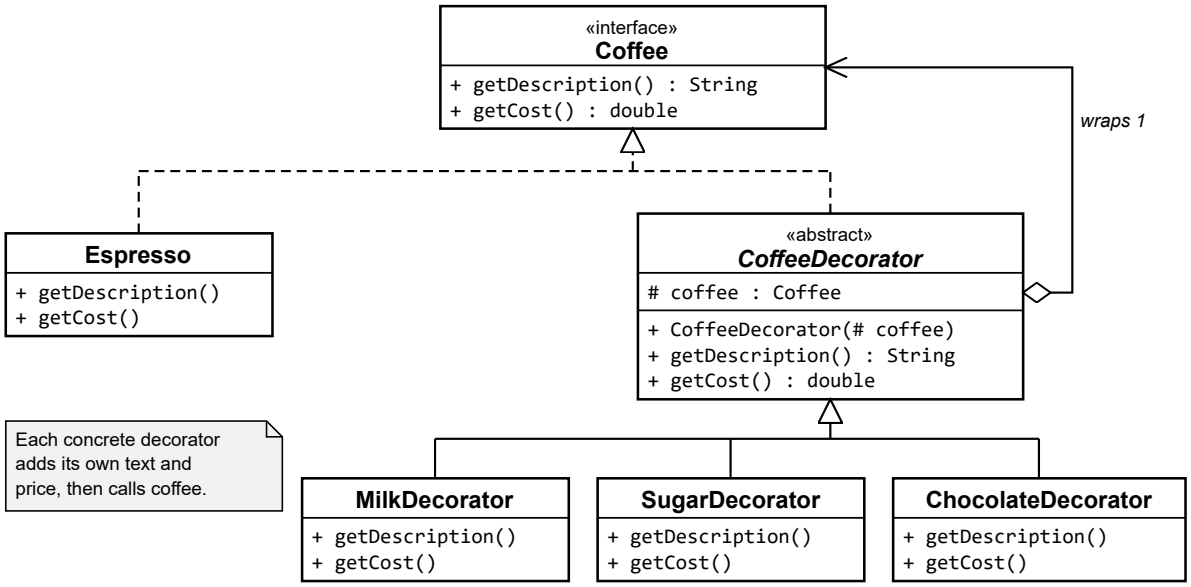
06 Decorator

Structural · “wrap the object”

Intent (GoF)	**Attach additional responsibilities to an object dynamically.** Decorators provide a flexible alternative to subclassing for extending functionality.
Use when	A base object can get **optional, combinable add-ons** and creating a subclass for every combination would explode (10 options ⇒ 2^{10} = **1024 subclasses**; n options ⇒ 2^n: 3⇒8, 15⇒32,768). Add features at **runtime** without changing the class.
Trigger words	“optional add-ons / extras / toppings / ingredients”, “without creating subclasses for every combination”, “price depends on base type + selected additions”, “instead of modifying the class directly or creating multiple subclasses”, “students may optionally add”, “bloated class with optional features”.
Participants	**Component** (interface) · **ConcreteComponent** (base object) · **Decorator** (abstract; implements Component AND **has-a** Component) · **ConcreteDecorators** (add behaviour before/after delegating).
Key ideas	Runtime extension · **composition over inheritance** · **Open/Closed** · flexible combinations · **each wrapper adds one responsibility**. Most important relationship: **composition (has-a)**.

UML — Coffee shop (lecture)



Decorator both **is-a** Coffee (dashed triangle: implements) and **has-a** Coffee (hollow diamond: aggregation, “wraps”). Concrete decorators **extend** the abstract decorator (solid triangle). `#` = protected.

Java

```
// 1. COMPONENT
public interface Coffee {
    String getDescription();
    double getCost();
}
// 2. CONCRETE COMPONENT
public class Espresso implements Coffee {
    public String getDescription() { return "Espresso"; }
    public double getCost() { return 350; }
}
// 3. ABSTRACT DECORATOR
public abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;                  // wrapped object
    public CoffeeDecorator(Coffee coffee) { this.coffee =
coffee; }
    public String getDescription() { return
coffee.getDescription(); }
    public double getCost() { return coffee.getCost(); }
}
```

```
// 4. CONCRETE DECORATORS
public class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee c) { super(c); }
    public String getDescription() { return
coffee.getDescription() + ", Milk"; }
    public double getCost() { return coffee.getCost() + 80; }
}
public class SugarDecorator extends CoffeeDecorator {
    public SugarDecorator(Coffee c) { super(c); }
    public String getDescription() { return
coffee.getDescription() + ", Sugar"; }
    public double getCost() { return coffee.getCost() + 20; }
}
// 5. CLIENT - wrap at runtime
Coffee coffee = new Espresso();
coffee = new MilkDecorator(coffee);
coffee = new SugarDecorator(coffee);
System.out.println(coffee.getDescription()); // Espresso, Milk,
Sugar
System.out.println(coffee.getCost());        // 450.0
```

Execution: `Sugar.getCost()` → calls `Milk.getCost()` → calls `Espresso.getCost()` =350 → Milk returns 350+80=430 → Sugar returns 430+20=**450**. Calls go **inward**, results come back **outward** (like layers of an onion). Pizza example: Plain Pizza 300 + Cheese 60 + Chicken 70 + Olives 40 = 470.

Advantages / disadvantages / challenge → mechanism

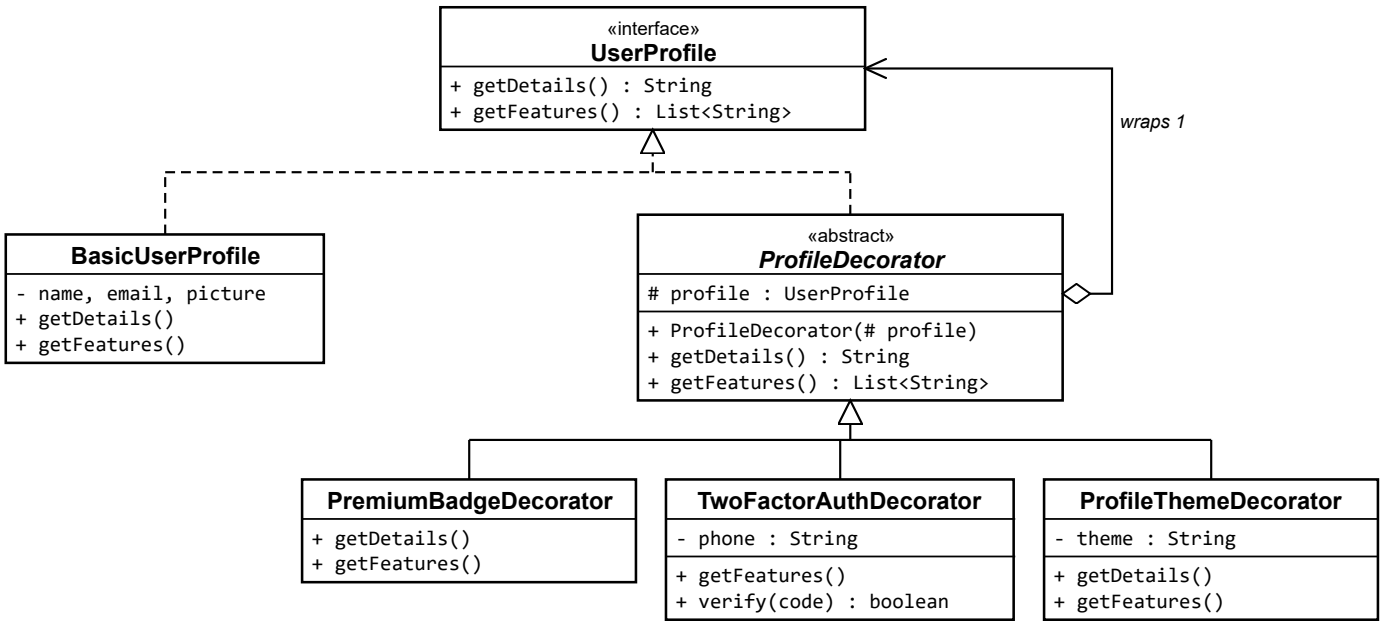
Advantages	Disadvantages	Challenge → mechanism
Runtime flexibility — add/remove features while running	**Many small classes** ⇒ complexity	Code complexity → one decorator = one responsibility, clear naming, a builder/factory to assemble common combos
No class explosion	**Harder debugging** (call passes through many wrappers)	Debugging → each decorator logs its step; keep chains short; unit-test each decorator alone
Open/Closed: existing structure untouched ⇒ no new bugs there	**Wrapping order matters** (discount then tax ≠ tax then discount)	Order/conflicts (two discounts, duplicate badge) → define a fixed wrapping order in the builder; reject duplicates
Reusable, small focused classes (SRP); incremental development — don't need to predict all features up front	More objects created; identity checks (`instanceof`) fail on the wrapped object	Performance (many objects, deep call chain) → negligible for small chains; cache computed cost; avoid very deep stacks

Decorator vs Inheritance: inheritance creates *classes* at compile time (one per combination, rigid); decorator creates *objects* at runtime by wrapping (flexible). **Decorator vs Proxy:** both wrap with the same interface; Decorator **adds responsibilities** and is stacked many times; Proxy **controls access**, usually one layer. **Abstract class vs interface:** abstract class = IS-A, shared attributes, `extends`; interface = CAN-DO contract, `implements`.

Questions & model answers

Social media `UserProfile` (name, email, picture); add premium badge, 2FA, profile theme without modifying the class or creating subclasses. Pattern? How applied? (3) Class structure (8). Code wrapping with multiple decorators (3). Challenges (4). *Quiz 5*

Decorator. Keep `UserProfile` as the component interface and `BasicUserProfile` as the concrete component. Create abstract `ProfileDecorator` that implements `UserProfile` and holds a `UserProfile`. Each feature is a concrete decorator that delegates to the wrapped profile and adds its part ("[Premium]" badge, 2FA verification, theme). A user's profile is built by wrapping only the features they enabled, at runtime.




```
public interface UserProfile { String getDetails(); List<String> getFeatures(); }

public class BasicUserProfile implements UserProfile {
    private String name, email, picture;
    public BasicUserProfile(String n, String e, String p) { name = n; email = e; picture = p; }
    public String getDetails() { return name + " (" + email + ")"; }
    public List<String> getFeatures() { return new ArrayList<>(List.of("Basic profile")); }
}
public abstract class ProfileDecorator implements UserProfile {
    protected UserProfile profile;
    public ProfileDecorator(UserProfile p) { this.profile = p; }
    public String getDetails() { return profile.getDetails(); }
    public List<String> getFeatures() { return profile.getFeatures(); }
}
public class PremiumBadgeDecorator extends ProfileDecorator {
    public PremiumBadgeDecorator(UserProfile p) { super(p); }
    public String getDetails() { return "[PREMIUM] " + profile.getDetails(); }
    public List<String> getFeatures() { List<String> f = profile.getFeatures(); f.add("Premium badge"); return f; }
}
public class TwoFactorAuthDecorator extends ProfileDecorator {
    private String phone;
```

```
    public TwoFactorAuthDecorator(UserProfile p, String phone) { super(p); this.phone = phone; }
    public List<String> getFeatures() { List<String> f = profile.getFeatures(); f.add("2FA via " + phone); return f; }
    public boolean verify(String code) { return code.equals(OtpService.send(phone)); }
}
public class ProfileThemeDecorator extends ProfileDecorator {
    private String theme;
    public ProfileThemeDecorator(UserProfile p, String theme) { super(p); this.theme = theme; }
    public String getDetails() { return profile.getDetails() + " | theme: " + theme; }
    public List<String> getFeatures() { List<String> f = profile.getFeatures(); f.add("Theme " + theme); return f; }
}
// CLIENT - wrapping with multiple decorators
UserProfile p = new BasicUserProfile("Nimal", "nimal@mail.com", "nimal.png");
p = new PremiumBadgeDecorator(p);
p = new TwoFactorAuthDecorator(p, "0771234567");
p = new ProfileThemeDecorator(p, "Dark");
System.out.println(p.getDetails());   // [PREMIUM] Nimal (nimal@mail.com) | theme: Dark
System.out.println(p.getFeatures());  // [Basic profile, Premium badge, 2FA via 077..., Theme Dark]
```

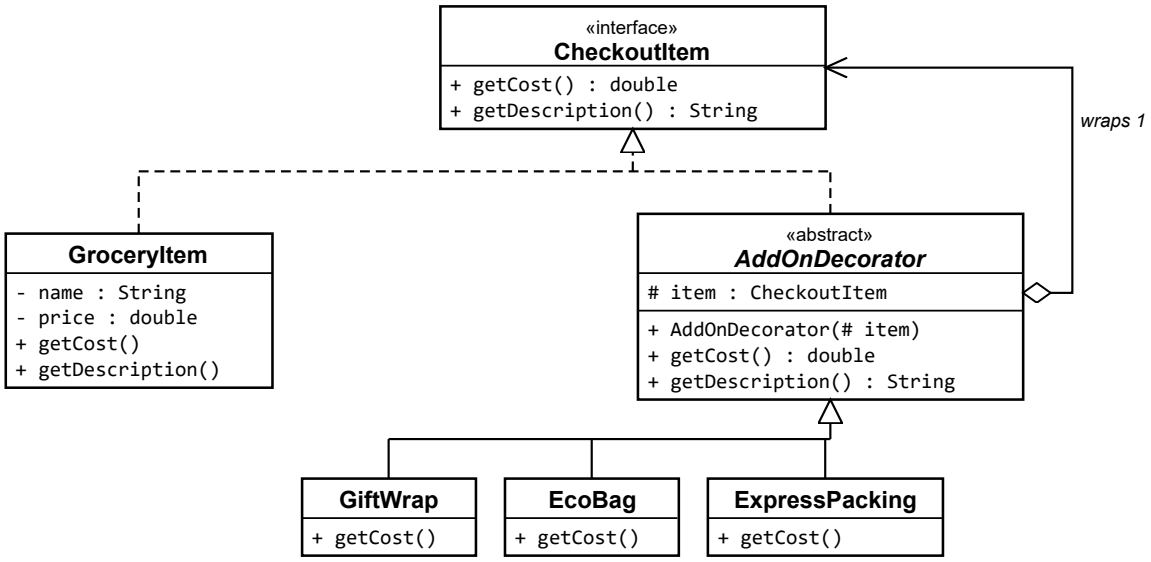
Challenges in real applications: Code complexity — many small decorator classes; harder for new developers to see the full feature set of one object. **Debugging** — a call travels through several wrappers; a bug may be in any layer; stack traces are deep. **Performance** — extra objects and an extra method call per layer (small, but grows with deep chains and many users). **Readability** — nested constructors (`new A(new B(new C(base)))`) are hard to read; **order** can change results; identity checks fail because the outer object isn't a `BasicUserProfile` . Mitigate with a builder/factory, logging per decorator, and documentation.

Coffee shop: base coffee + optional milk, sugar, chocolate; price calculated dynamically without a subclass hierarchy. Best pattern; Coffee interface with `getCost()` / `getDescription()` ; BasicCoffee class. If each coffee has a different profit, how to extend? *2024 Q1.1–1.4 (20); 2023 Q3.1–3.2*

Decorator (as UML/code above; `BasicCoffee` = `Espresso`). **Different profit per coffee:** add `getProfit()` to the `Coffee` interface: each base coffee returns its own margin and each decorator returns `coffee.getProfit() + addOnProfit` ; selling price = `getCost() + getProfit()` . Alternatively, a `ProfitMarginDecorator(coffee, rate)` wraps the finished drink and returns `coffee.getCost() * (1 + rate)` — no existing class changes (OCP). Irish Coffee / Nitro Coffee = different concrete components (or base + decorators: whiskey, sugar, cream).

```
public class BasicCoffee implements Coffee {
    public String getDescription() { return "Basic coffee"; }
    public double getCost() { return 200.0; }
}
public class ProfitMarginDecorator extends CoffeeDecorator {
    private final double rate;
    public ProfitMarginDecorator(Coffee c, double rate) { super(c); this.rate = rate; }
    public double getCost() { return coffee.getCost() * (1 + rate); }
}
```

Self-checkout kiosk: optional add-ons (gift-wrap, eco-bag, express packing) — define `CheckoutItem` and `GroceryItem` . *2025 Q1.1(b), 1.2–1.3 (10)*



```
public interface CheckoutItem {
    double getCost();
    String getDescription();
}
public class GroceryItem implements CheckoutItem {
    private String name; private double price;
    public GroceryItem(String n, double p) { name = n; price = p; }
    public double getCost() { return price; }
    public String getDescription() { return name; }
}
public class GiftWrap extends AddOnDecorator {
    public GiftWrap(CheckoutItem i) { super(i); }
    public double getCost() { return item.getCost() + 150; }
    public String getDescription() { return item.getDescription() + " + gift wrap"; }
}
CheckoutItem it = new GiftWrap(new EcoBag(new GroceryItem("Rice 5kg", 1200)));
```

Library: optional Extended Borrow Period, SMS Notification, Email Receipt without subclasses per combination. *2025 Q3.1(d)* · **Cake ordering: decorations and colour change price and delivery time.** *2022 Q4.1*

Decorator on `BorrowTransaction` (`ExtendedPeriodDecorator` adds days + fee, `SmsNotificationDecorator` sends SMS after `process()`, `EmailReceiptDecorator`). Cake: `Cake` interface with `getPrice()` and `getDeliveryDays()`; `FondantDecoration`, `ColourDecorator`… each add price and days.

Online course: base features (content, assignments, final assessment); students may optionally add AI learning assistant, premium materials and one-to-one mentor support. Explain (no UML/code needed). *Quiz 6 Q5 (3)*

Decorator. `Course` interface (`getFeatures()`, `getPrice()`); `BaseCourse` gives content, assignments, final assessment. Abstract `CourseDecorator` wraps a `Course`; `AIAssistantDecorator`, `PremiumMaterialsDecorator`, `MentorSupportDecorator` each add their feature and fee and delegate the rest. At enrolment, wrap only the options the student chose, e.g. `new MentorSupport(new AIAssistant(new ProgrammingCourse()))`. The Template Method completion flow (Note 07) is unchanged.

Mistakes: decorator not implementing the component interface · decorator creating its own base instead of receiving it in the constructor · forgetting to delegate to the wrapped object (loses earlier add-ons) · drawing inheritance instead of has-a from decorator to component · using subclasses `EspressoWithMilkAndSugar`.